

Tasca S2.01. Nocions bàsiques SQL

Nivell 1

Exercici 1 - A partir dels documents adjunts (estructura_dades i dades_introduir), importa les dues taules. Mostra les característiques principals de l'esquema creat i explica les diferents taules i variables que existeixen. Assegura't d'incloure un diagrama que il·lustri la relació entre les diferents taules i variables.

El archivo 'estructura_dades.sql' tiene como principal objetivo crear una base de datos llamada 'transactions' y, dentro de esta base de datos, crea dos tablas para la información de las empresas y de las transacciones:

Tabla 'company', contiene la información de las empresas en variables de tipo 'varchar' y tiene como columnas:

- id: la primary key que identifica la empresa;
- company_name: nombre de la empresa;
- phone: número de teléfono;
- email: dirección de correo electrónico;
- country: país de la empresa;
- website: sitio web de la empresa.

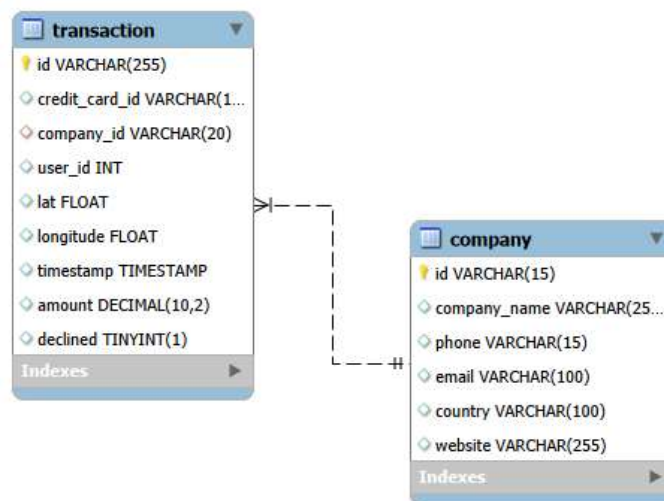
Tabla 'transaction', contiene la información de las transacciones realizadas y tiene como columnas:

- id: la primary key de la tabla que identifica la transacción, está almacenada como 'varchar';
- credit_card_id: tarjeta de crédito utilizada en la operación, que es una foreign key, haciendo referencia a una tabla credit_card que todavía no existe, también es una variable 'varchar';
- company_id: código de la empresa relacionada con la operación, es una foreign key de la tabla company, también es una variable 'varchar';
- user_id: identificación del usuario, de tipo 'int' o sea número entero, es una foreign key de una tabla user, que todavía no existe;
- lat: latitud donde se realizó la transacción de tipo 'float' o número real;
- longitude: longitud donde se realizó la transacción de tipo 'float' o número real;
- timestamp: fecha y hora donde se realizó la transacción, de tipo 'timestamp';

- amount: valor de la operación, con dos casas decimales;
- declined: indica si la transacción fue denegada. Es de tipo boolean, avalia si es verdadero o falso: para valores 0 es falso y la operación fue aprobada, para valores 1 es verdadero y la operación fue denegada.

El archivo 'dades_introducir.sql' tiene como objetivo introducir los datos en las tablas creadas anteriormente. En la tabla 'company' se introducen las informaciones de 100 empresas y en la tabla 'transaction' son inseridos los registros de 100 mil operaciones.

La relación entre las dos tablas es de 1 a N (1 a muchos), o sea, una empresa puede tener registros de muchas transacciones, pero una transacción ocurre en apenas una empresa.



NT-Ex_1_estructura dades NT-Ex_1_dades_introducir

```

1 USE transactions;
2
3 -- Insertamos datos de company
4 INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
5 INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
6 INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
7 INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
8 INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
9 INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
10 INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
11 INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
12 INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
13 INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
14 INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
15 INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
16 INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
17 INSERT INTO company (id, company_name, phone, email, country, website) VALUES (

```

Output

#	Time	Action	Message	Duration / Fetch
100104	11:20:12	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, a...	1 row(s) affected	0.016 sec
100105	11:20:12	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, a...	1 row(s) affected	0.000 sec
100106	11:20:12	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, a...	1 row(s) affected	0.000 sec
100107	11:20:12	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, a...	1 row(s) affected	0.000 sec

Exercici 2 - Utilitzant JOIN realitzaràs les següents consultes:

- Llistat dels països que estan generant vendes.

Como el ejercicio pide identificar los países que están generando ventas, se consideran las transacciones no declinadas. Además, se utiliza SELECT DISTINCT para evitar que un mismo país aparezca repetido.

The screenshot shows a database IDE window titled 'Tasca S2.01. Nocions bàsiques...'. The SQL editor contains the following code:

```
7 • USE transactions;
8
9 /* Exercici 2 - Utilitzant JOIN realitzaràs les següents consultes:
10
11 Llistat dels països que estan generant vendes. */
12 • SELECT DISTINCT c.country AS lista_paises
13 FROM company c
14 JOIN transaction t ON c.id = t.company_id
15 WHERE t.declined = 0;
```

The 'Result Grid' shows the results of the query:

lista_paises
Germany
Norway
France
Italy
United Kingdom

The 'Output' pane shows the execution log:

#	Time	Action	Message	Duration / Fetch
100106	11:20:12	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timesta...	1 row(s) affected	0.000 sec
100107	11:20:12	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timesta...	1 row(s) affected	0.000 sec
100108	11:30:05	USE transactions	0 row(s) affected	0.000 sec
100109	11:30:44	SELECT DISTINCT c.country AS lista_paises FROM company c JOIN transaction t ON c...	15 row(s) returned	0.328 sec / 0.000 sec

- Des de quants països es generen les vendes.

Aquí sigue el mismo racional de la pregunta anterior, pero ahora tenemos el número total de países que generan ventas.

The screenshot shows the same database IDE window. The SQL editor contains the following code:

```
17 /* Des de quants països es generen les vendes. */
18 • SELECT COUNT(DISTINCT c.country) AS contagem_paises
19 FROM company c
20 JOIN transaction t ON c.id = t.company_id
21 WHERE t.declined = 0;
22
23
24
25
```

The 'Result Grid' shows the results of the query:

contagem_paises
15

The 'Output' pane shows the execution log:

#	Time	Action	Message	Duration / Fetch
100108	11:30:05	USE transactions	0 row(s) affected	0.000 sec
100109	11:30:44	SELECT DISTINCT c.country AS lista_paises FROM company c JOIN transaction t ON c...	15 row(s) returned	0.328 sec / 0.000 sec
100110	11:34:48	SELECT COUNT(DISTINCT c.country) AS count_paises FROM company c JOIN transa...	1 row(s) returned	0.328 sec / 0.000 sec
100111	11:35:24	SELECT COUNT(DISTINCT c.country) AS contagem_paises FROM company c JOIN tra...	1 row(s) returned	0.281 sec / 0.000 sec

- Identifica la companyia amb la mitjana més gran de vendes.

Como se necesita calcular la media de ventas para cada compañía, se utiliza GROUP BY agrupando por 'c.id' y 'c.company_name'. Se incluye el id porque identifica de forma única cada compañía, incluso si hubiera empresas con el mismo nombre. Además, utilicé el ORDER BY DESC y LIMIT para que solo se presente la empresa con la media más grande.

```

23 # Identifica la companyia amb la mitjana més gran de vendes.
24 SELECT c.id, c.company_name, ROUND(AVG(t.amount),2) AS media_ventas
25 FROM company c
26 JOIN transaction t ON c.id = t.company_id
27 WHERE t.declined = 0
28 GROUP BY c.id, c.company_name
29 ORDER BY media_ventas DESC
30 LIMIT 1;
31

```

id	company_name	media_ventas
b-2222	Ac Fermentum Incorporated	284.91

Result 6 x

Output

#	Time	Action	Message	Duration / Fetch
100112	11:37:04	SELECT c.id, c.company_name, AVG(t.amount) AS media_ventas FROM company c JOI...	1 row(s) returned	0.515 sec / 0.000 sec
100113	11:38:19	SELECT c.id, c.company_name, ROUND(AVG(t.amount),2) AS media_ventas FROM com...	Error Code: 1054. Unknown column 'media_ventas' in 'order clause'	0.016 sec
100114	11:38:27	SELECT c.id, c.company_name, ROUND(AVG(t.amount),2) AS media_ventas FROM com...	1 row(s) returned	0.453 sec / 0.000 sec
100115	11:38:31	SELECT c.id, c.company_name, ROUND(AVG(t.amount),2) AS media_ventas FROM com...	1 row(s) returned	0.422 sec / 0.000 sec

Exercici 3 - Utilitzant només subconsultes (sense utilitzar JOIN):

- Mostra totes les transaccions realitzades per empreses d'Alemanya.

Para este ejercicio, la subconsulta fue hecha para filtrar los ids de las empresas de Alemania y utilizarlos en el filtro WHERE.

```

32 /* Exercici 3 - Utilitzant només subconsultes (sense utilitzar JOIN):
33
34 Mostra totes les transaccions realitzades per empreses d'Alemanya. */
35 SELECT *
36 FROM transaction
37 WHERE company_id IN(SELECT id
38 FROM company
39 WHERE country = 'Germany');
40

```

id	credit_card_id	company_id	user_id	lat	longitude	timestamp	amount	declined
00138038-206D-4C03-94B7-63A2676EB984	CcS-4899	b-2222	318	41.3781	12.447	2020-03-25 10:43:43	426.36	0
0013C1B6-3B84-4D6C-8154-E2B3FEBCA8E9	CcS-5070	b-2222	489	41.3814	2.18176	2020-12-17 18:15:37	316.90	0
00201A11-2E62-44C4-941D-198FC80B77F0	CcU-3512	b-2222	193	55.5704	-3.65129	2021-01-22 23:44:27	453.04	0
00235618-0A5C-4D49-9DCB-B3A9405D8923	CcS-8137	b-2222	3556	59.8421	18.729	2020-09-09 15:43:19	263.14	0
005A5A7B-1F1A-4B6C-9B15-1625A78C9C38	CcS-8998	b-2222	4417	41.1591	-8.63905	2024-05-15 09:10:11	442.01	0

transaction 7 x

Output

#	Time	Action	Message	Duration / Fetch
100113	11:38:19	SELECT c.id, c.company_name, ROUND(AVG(t.amount),2) AS media_ventas FROM com...	Error Code: 1054. Unknown column 'media_ventas' in 'order clause'	0.016 sec
100114	11:38:27	SELECT c.id, c.company_name, ROUND(AVG(t.amount),2) AS media_ventas FROM com...	1 row(s) returned	0.453 sec / 0.000 sec
100115	11:38:31	SELECT c.id, c.company_name, ROUND(AVG(t.amount),2) AS media_ventas FROM com...	1 row(s) returned	0.422 sec / 0.000 sec
100116	11:54:22	SELECT * FROM transaction WHERE company_id IN(SELECT id FROM company WHE...	13291 row(s) returned	0.015 sec / 0.125 sec

- Llista les empreses que han realitzat transaccions per un amount superior a la mitjana de totes les transaccions.

En este caso, fue necesario dos subconsultas: la primera subconsulta calcula la media del 'amount' de todas las transacciones. La segunda obtiene los company_id de las transacciones cuyo 'amount' es superior a esa media. Finalmente, la consulta principal muestra el nombre de las empresas correspondientes.

The screenshot shows a SQL query in SQL Studio. The query is as follows:

```

41  /*Llista les empreses que han realitzat transaccions per un amount superior a la mitjana de totes les transaccions. */
42  SELECT company_name
43  FROM company
44  WHERE id IN (SELECT DISTINCT company_id
45              FROM transaction
46              WHERE amount > (SELECT AVG(amount)
47                             FROM transaction));
48
49

```

The Result Grid shows the following companies:

company_name
Ac Fermentum Incorporated
Magna A Neque Industries
Fusce Corp.
Convallis In Incorporated
Ante Iaculis Nec Foundation

The Output pane shows the execution log:

#	Time	Action	Message	Duration / Fetch
100114	11:38:27	SELECT c.id, c.company_name, ROUND(AVG(t.amount),2) AS media_ventas FROM com...	1 row(s) returned	0.453 sec / 0.000 sec
100115	11:38:31	SELECT c.id, c.company_name, ROUND(AVG(t.amount),2) AS media_ventas FROM com...	1 row(s) returned	0.422 sec / 0.000 sec
100116	11:54:22	SELECT * FROM transaction WHERE company_id IN(SELECT id FROM company WHE...	13291 row(s) returned	0.015 sec / 0.125 sec
100117	11:57:50	SELECT company_name FROM company WHERE id IN (SELECT DISTINCT company_id...	100 row(s) returned	0.156 sec / 0.000 sec

- Eliminarian del sistema les empreses que no tenen transaccions registrades, entrega el llistat d'aquestes empreses.

Para contestar esta pregunta, realizamos una subconsulta para obtener las empresas que tienen transacciones registradas y luego buscamos las empresas que no aparecen en la lista mediante NOT IN.

The screenshot shows a SQL query in SQL Studio. The query is as follows:

```

49  /* Eliminaran del sistema les empreses que no tenen transaccions registrades, entrega el llistat d'aquestes empreses. */
50  SELECT company_name
51  FROM company
52  WHERE id NOT IN (SELECT DISTINCT company_id
53                  FROM transaction);
54
55
56
57

```

The Result Grid shows the following companies:

company_name

The Output pane shows the execution log:

#	Time	Action	Message	Duration / Fetch
100121	12:10:13	SELECT id FROM transaction	100000 row(s) returned	0.000 sec / 0.062 sec
100122	12:10:22	SELECT company_id FROM transaction	100000 row(s) returned	0.000 sec / 0.031 sec
100123	12:10:31	SELECT DISTINCT company_id FROM transaction	100 row(s) returned	0.000 sec / 0.000 sec
100124	12:10:54	SELECT company_name FROM company WHERE id NOT IN (SELECT DISTINCT comp...	0 row(s) returned	0.046 sec / 0.000 sec

Exercici 4 - La teva tasca és dissenyar i crear una taula anomenada "credit_card" que emmagatzemi detalls crucials sobre les targetes de crèdit. La nova taula ha de ser capaç d'identificar de manera única cada targeta i establir una relació adequada amb les altres dues taules ("transaction" i "company"). Després de crear la taula serà necessari que ingressis la informació del document denominat "dades_introduir_credit". Recorda mostrar el diagrama i realitzar una breu descripció d'aquest.

La tabla 'credit_card' contiene la información de las tarjetas de crédito y todas sus columnas están definidas como 'varchar':

- id: la primary key que identifica la tarjeta de crédito;
- iban: numero de la cuenta asociada a la tarjeta de crédito;
- pan: número de la tarjeta;
- pin: pin de la tarjeta;
- cvv: código de seguridad;
- expiring_date: fecha de vencimiento de la tarjeta.

```

60 CREATE TABLE IF NOT EXISTS credit_card (
61     id VARCHAR(15) PRIMARY KEY,
62     iban VARCHAR(255),
63     pan VARCHAR(50),
64     pin VARCHAR(50),
65     cvv VARCHAR(50),
66     expiring_date VARCHAR(50)
67 );

```

#	Time	Action	Message	Duration / Fetch
100122	12:10:22	SELECT company_id FROM transaction	100000 row(s) returned	0.000 sec / 0.031 sec
100123	12:10:31	SELECT DISTINCT company_id FROM transaction	100 row(s) returned	0.000 sec / 0.000 sec
100124	12:10:54	SELECT company_name FROM company WHERE id NOT IN (SELECT DISTINCT comp...	0 row(s) returned	0.046 sec / 0.000 sec
100125	12:43:53	CREATE TABLE IF NOT EXISTS credit_card (id VARCHAR(15) PRIMARY KEY, iban VA...	0 row(s) affected	0.062 sec

Después de insertar los datos a través del archivo 'dades_introduir_credit.sql', fue necesario también crear la FK con la tabla 'transaction':

```

68
69 ALTER TABLE transaction
70 ADD CONSTRAINT fk_credit_card_id
71 FOREIGN KEY (credit_card_id)
72 REFERENCES credit_card(id);

```

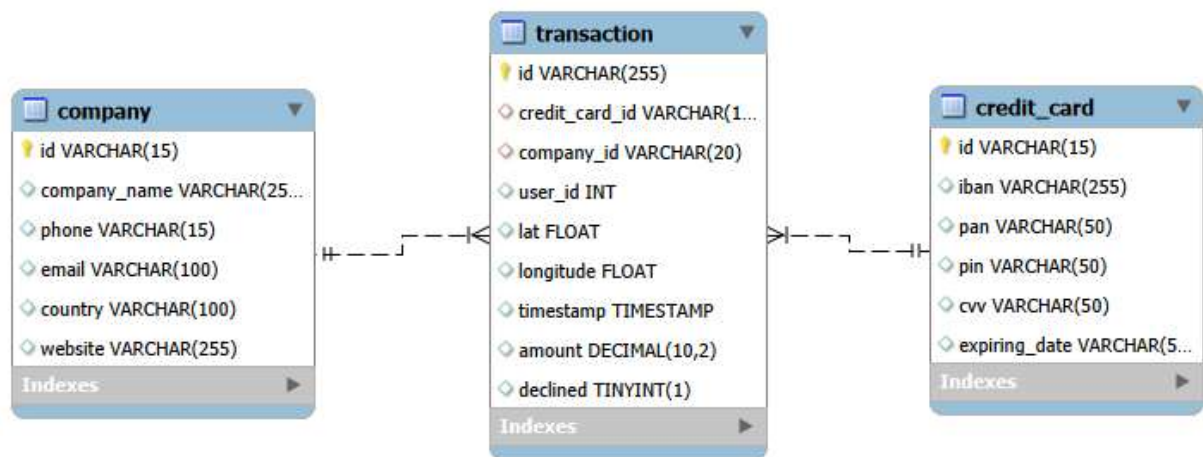
#	Time	Action	Message	Duration / Fetch
105124	12:54:39	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9579', 'XX...	1 row(s) affected	0.000 sec
105125	12:54:39	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9580', 'XX...	1 row(s) affected	0.015 sec
105126	12:54:39	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9581', 'XX...	1 row(s) affected	0.000 sec
105127	12:55:00	ALTER TABLE transaction ADD CONSTRAINT fk_credit_card_id FOREIGN KEY (credit_c...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0	3.984 sec

La tabla se relaciona con la tabla 'transaction' por la columna id de 'credit_card' y la columna 'credit_card_id' de 'transaction'. La relación es de 1 a N, o sea, una tarjeta puede estar asociada a muchas transacciones mientras una transacción está asociada a solo una tarjeta a la vez.

credit_card (1) ————— (N) transaction

La relación con la tabla 'company' no es directa, ocurre a través de la relación con la tabla 'transaction'. Una empresa puede tener muchas transacciones y cada transacción está asociada a una empresa mediante 'company_id'.

credit_card (1) ————— (N) transaction (N) ————— (1) company



Exercici 5 - El departament de Recursos Humans ha identificat un error en el número de compte associat a la targeta de crèdit amb ID CcU-2938. La informació que ha de mostrar-se per a aquest registre és: TR323456312213576817699999. Recorda mostrar que el canvi es va realitzar.

Antes:

```

73
74 /*Exercici 5 - El departament de Recursos Humans ha identificat un error en el número de compte associat a la targeta de crèdit amb ID CcU-2938.
75 La informació que ha de mostrar-se per a aquest registre és: TR323456312213576817699999. Recorda mostrar que el canvi es va realitzar.*/
76 SELECT *
77 FROM credit_card
78 WHERE id = 'CcU-2938';
79
80
81

```

id	iban	pan	pin	cvv	expiring_date
CcU-2938	TR301950312213576817638661	5424465566813633	3257	984	10/30/22

credit card 16 x

Output

#	Time	Action	Message	Duration / Fetch
105125	12:54:39	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9580', 'X...', 'X...', 'X...', 'X...')	1 row(s) affected	0.015 sec
105126	12:54:39	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9581', 'X...', 'X...', 'X...', 'X...')	1 row(s) affected	0.000 sec
105127	12:55:00	ALTER TABLE transaction ADD CONSTRAINT fk_credit_card_id FOREIGN KEY (credit_card_id) REFERENCES credit_card (id)	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0	3.984 sec
105128	13:00:34	SELECT * FROM credit_card WHERE id = 'CcU-2938'	1 row(s) returned	0.000 sec / 0.000 sec

Utilizamos el comando UPDATE para modificar el número de cuenta (iban) asociado a la tarjeta con ID CcU-2938. Finalmente, realizamos una consulta para comprobar que el cambio se ha realizado correctamente.

The screenshot shows a database management interface with the following SQL commands and results:

```

80 • UPDATE credit_card
81   SET iban = 'TR323456312213576817699999'
82   WHERE id = 'CcU-2938';
83
84 • SELECT *
85   FROM credit_card
86   WHERE id = 'CcU-2938';
87
88
89

```

The Result Grid shows the following data for the credit_card table:

id	iban	pan	pin	cvv	expiring_date
CcU-2938	TR323456312213576817699999	5424465566813633	3257	984	10/30/22

The Output section shows the following actions and messages:

#	Time	Action	Message	Duration / Fetch
105127	12:55:00	ALTER TABLE transaction ADD CONSTRAINT fk_credit_card_id FOREIGN KEY (credit_...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0	3.984 sec
105128	13:00:34	SELECT * FROM credit_card WHERE id = 'CcU-2938'	1 row(s) returned	0.000 sec / 0.000 sec
105129	13:01:31	UPDATE credit_card SET iban = 'TR323456312213576817699999' WHERE id = 'CcU-2...	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.016 sec
105130	13:01:40	SELECT * FROM credit_card WHERE id = 'CcU-2938'	1 row(s) returned	0.000 sec / 0.000 sec

Exercici 6 - En la taula "transaction" ingressa una nova transacció amb la següent informació:

- Id = 108B1D1D-5B23-A76C-55EF-C568E49A99DD
- credit_card_id = CcU-9999
- company_id = b-9999
- user_id = 9999
- lat = 829.999
- longitude = -117.999
- amount = 111.11
- declined = 0

No es posible insertar inicialmente la nueva transacción porque los valores 'b-9999' y 'CcU-9999' no existen en las tablas 'company' y 'credit_card', respectivamente.

Para poder insertar la nueva transacción, fue necesario crear previamente los registros correspondientes en las tablas referenciadas por las claves foráneas, ya que estos valores no existían en las tablas padre.

Tasca S2.01. Nocions bàsiques...

```

88 /*Exercici 6
89 En la taula "transaction" ingressa una nova transacció amb la següent informació:
90 Id = 10881D1D-5B23-A76C-55EF-C568E49A990D
91 credit_card_id = CcU-9999
92 company_id = b-9999
93 user_id = 9999
94 lat = 829.999
95 longitude = -117.999
96 amount = 111.11
97 declined = 0 */
98 • INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, amount, declined) VALUES ('10881D1D-5B23-A76C-55EF-C568E49A990D', '1
99
100
101
102
103
104

```

Output

#	Time	Action	Message	Duration / Fetch
105128	13:00:34	SELECT * FROM credit_card WHERE id = 'CcU-2938'	1 row(s) returned	0.000 sec / 0.000 sec
105129	13:01:31	UPDATE credit_card SET iban = 'TR323456312213576817699999' WHERE id = 'CcU-2...	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.016 sec
105130	13:01:40	SELECT * FROM credit_card WHERE id = 'CcU-2938'	1 row(s) returned	0.000 sec / 0.000 sec
105131	13:07:13	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, amount, declined) VALUES ('10881D1D-5B23-A76C-55EF-C568E49A990D', '1	Error Code: 1452. Cannot add or update a child row: a foreign key constraint fails (transac...	0.015 sec

Tasca S2.01. Nocions bàsiques...

```

94 lat = 829.999
95 longitude = -117.999
96 amount = 111.11
97 declined = 0 */
98 • INSERT INTO company (id, company_name, phone, email, country, website) VALUES ('b-9999', 'Bulls company', '06 66 66 55 22', 'bulls@icloud.com', 'Aust
99 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-9999', 'TR301940312213576817638661', '5424466566813633', '6666', '666',
100 • INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, amount, declined) VALUES ('10881D1D-5B23-A76C-55EF-C568E49A990D', '1
101
102
103
104
105
106
107
108
109
110

```

Output

#	Time	Action	Message	Duration / Fetch
105132	13:14:27	INSERT INTO company (id, company_name, phone, email, country, website) VALUES (b-...	1 row(s) affected	0.015 sec
105133	13:14:30	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timesta...	Error Code: 1452. Cannot add or update a child row: a foreign key constraint fails (transac...	0.016 sec
105134	13:15:46	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-9999', 'T...	1 row(s) affected	0.000 sec
105135	13:15:59	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, amount,...	1 row(s) affected	0.000 sec

Exercici 7 - Des de recursos humans et sol·liciten eliminar la columna "pan" de la taula credit_card. Recordar mostrar el canvi realitzat.

Para realizar la modificación solicitada, utilizamos ALTER TABLE junto con DROP COLUMN para eliminar la columna pan de la tabla credit_card. Finalmente, realizamos un SELECT para comprobar que la columna ha sido eliminada.

Tasca S2.01. Nocions bàsiques.

102 # Exercici 7 - Des de recursos humans et sol·liciten eliminar la columna "pan" de la taula credit_card. Recordar mostrar el canvi realitzat.

103 • ALTER TABLE credit_card

104 DROP COLUMN pan;

105

106 • SELECT *

107 FROM credit_card;

108

109

110

111

id	iban	pin	cvv	expiring_date
CcS-4857	XX4857591835292505850771	1819	467	09/27/25
CcS-4858	XX8581768137002436094025	3964	817	12/28/28
CcS-4859	XX7826930491423553609370	4983	277	11/26/26
CcS-4860	XX5559590368835304645299	6876	661	07/27/27
CcS-4861	XX2035182877195191627307	5710	398	04/25/26

credit_card 18

Output

#	Time	Action	Message	Duration / Fetch
105134	13:15:46	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-9999', T...	1 row(s) affected	0.000 sec
105135	13:15:59	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, amount, ...	1 row(s) affected	0.000 sec
105136	13:22:53	ALTER TABLE credit_card DROP COLUMN pan	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.063 sec
105137	13:22:55	SELECT * FROM credit_card	5001 row(s) returned	0.000 sec / 0.000 sec

Exercici 8 - Descarrega els arxius CSV que trobaràs a l'apartat de recursos:

- american_users.csv
- european_users.csv
- companies.csv
- credit_cards.csv
- transactions.csv

Estudia'ls i dissenya una base de dades amb un esquema d'estrella que contingui, almenys 4 taules de les quals puguem realitzar les següents consultes:

La taula de products.csv l'utilitzarem més endavant.

Aunque la base de datos sea muy similar, elegí crearla desde cero con el nombre 'transactions_star', para contemplar las nuevas columnas en 'transaction', 'company' y 'credit_card' y la nueva tabla 'user' que es una combinación de las tablas de usuarios americanos y europeos. Yo observe que ambas las tablas tenían la misma estructura y por eso opte por crear una nueva columna 'region' para hacer la distinción de los usuarios. La tabla de hechos y la tabla que conecta todas las demás tablas es 'transaction', pero vale decir que 'user' y 'credit_card' también mantienen relación directa.

Vale comentar que el orden de ejecución importa, empecé por las empresas, después usuarios, tarjeta de crédito (que tiene FK user_id) y por fin las transacciones, que tiene FK con todas las demás tablas.

Creando la base de datos:

The screenshot shows the SQL Developer interface with the 'Navigator' pane on the left displaying a list of schemas. The main editor pane shows a SQL script for creating the 'transactions_star' database. The script includes comments in Spanish and SQL commands to create the database and use it.

```
109 /*Ejercicio 8 - Descarga els arxius CSV que trobaràs a l'apartat de recursos:
110 o american_users.csv
111 o european_users.csv
112 o companies.csv
113 o credit_cards.csv
114 o transactions.csv
115 Estudia'ls i dissenya una base de dades amb un esquema d'estrella que contingui, almenys 4 taules de les quals puguis realitzar les
116 següents consultes:
117
118 La taula de products.csv l'utilitzarem més endavant.*/
119
120 -- Primero, creamos la base de datos
121 CREATE DATABASE IF NOT EXISTS transactions_star;
122
123 USE transactions_star;
124
125
```

The 'Output' pane at the bottom shows the execution results:

#	Time	Action	Message	Duration / Fetch
1	10:25:47	USE transactions_star	Error Code: 1049. Unknown database 'transactions_star'	0.047 sec
2	10:25:54	CREATE DATABASE IF NOT EXISTS transactions_star	1 row(s) affected	0.047 sec
3	10:25:57	USE transactions_star	0 row(s) affected	0.000 sec

Creando la tabla 'company':

The screenshot shows the SQL Developer interface with the 'Navigator' pane on the left. The main editor pane shows a SQL script for creating the 'company' and 'user' tables. The script includes comments in Spanish and SQL commands to create the tables.

```
125 -- Ahora vamos crear 4 tablas: company, user, credit_card y transaction
126
127 -- Tabla company
128 CREATE TABLE IF NOT EXISTS company (
129 company_id VARCHAR(15) PRIMARY KEY,
130 company_name VARCHAR(255),
131 phone VARCHAR(20),
132 email VARCHAR(100),
133 country VARCHAR(100),
134 website VARCHAR(255),
135 merchant_category VARCHAR(100),
136 merchant_price_position VARCHAR(100)
137 );
138
139 -- Tabla user
140 CREATE TABLE IF NOT EXISTS user (
141 id INT PRIMARY KEY,
```

The 'Output' pane at the bottom shows the execution results:

#	Time	Action	Message	Duration / Fetch
1	10:25:47	USE transactions_star	Error Code: 1049. Unknown database 'transactions_star'	0.047 sec
2	10:25:54	CREATE DATABASE IF NOT EXISTS transactions_star	1 row(s) affected	0.047 sec
3	10:25:57	USE transactions_star	0 row(s) affected	0.000 sec
4	10:27:51	CREATE TABLE IF NOT EXISTS company (company_id VARCHAR(15) PRIMARY KEY, ...	0 row(s) affected	0.609 sec

Creando la tabla 'user':

The screenshot shows the SQL Developer interface with the 'Navigator' pane on the left. The main editor pane shows a SQL script for creating the 'user' table. The script includes comments in Spanish and SQL commands to create the table.

```
139 -- Tabla user
140 CREATE TABLE IF NOT EXISTS user (
141 id INT PRIMARY KEY,
142 name VARCHAR(50),
143 surname VARCHAR(50),
144 phone VARCHAR(50),
145 email VARCHAR(100),
146 birth_date VARCHAR(50),
147 country VARCHAR(100),
148 city VARCHAR(100),
149 postal_code VARCHAR(50),
150 address VARCHAR(100),
151 signup_date VARCHAR(50),
152 user_segment VARCHAR(50),
153 income_band VARCHAR(50)
154 );
155
```

The 'Output' pane at the bottom shows the execution results:

#	Time	Action	Message	Duration / Fetch
2	10:25:54	CREATE DATABASE IF NOT EXISTS transactions_star	1 row(s) affected	0.047 sec
3	10:25:57	USE transactions_star	0 row(s) affected	0.000 sec
4	10:27:51	CREATE TABLE IF NOT EXISTS company (company_id VARCHAR(15) PRIMARY KEY, ...	0 row(s) affected	0.609 sec
5	10:28:40	CREATE TABLE IF NOT EXISTS user (id INT PRIMARY KEY, name VARCHAR(50), ...	0 row(s) affected	0.031 sec

Creando la tabla 'credit_card':

The screenshot shows a SQL IDE window titled 'Tasca S2.01. Nociones básicas...'. The left sidebar shows a list of schemas, with 'transactions_star' selected. The main editor displays the following SQL code:

```
-- Tabla credit_card
157 CREATE TABLE IF NOT EXISTS credit_card (
158     id VARCHAR(15) PRIMARY KEY,
159     user_id INT REFERENCES user(id),
160     iban VARCHAR(50),
161     pan VARCHAR(50),
162     pin VARCHAR(10),
163     cvv VARCHAR(10),
164     track1 VARCHAR(100),
165     track2 VARCHAR(100),
166     expiring_date VARCHAR(10),
167     card_type VARCHAR(50),
168     card_renewal_flag BOOLEAN
169 );
```

The bottom output pane shows the execution results:

#	Time	Action	Message	Duration / Fetch
3	10:25:57	USE transactions_star	0 row(s) affected	0.000 sec
4	10:27:51	CREATE TABLE IF NOT EXISTS company (company_id VARCHAR(15) PRIMARY KEY, ...	0 row(s) affected	0.609 sec
5	10:28:40	CREATE TABLE IF NOT EXISTS user (id INT PRIMARY KEY, name VARCHAR(50), ...	0 row(s) affected	0.031 sec
6	10:29:45	CREATE TABLE IF NOT EXISTS credit_card (id VARCHAR(15) PRIMARY KEY, user_id IN...	0 row(s) affected	0.031 sec

Creando la tabla 'transaction':

The screenshot shows the same SQL IDE window. The left sidebar shows the 'transactions' schema selected. The main editor displays the following SQL code:

```
-- Tabla transaction
172 CREATE TABLE IF NOT EXISTS transaction (
173     id VARCHAR(255) PRIMARY KEY,
174     card_id VARCHAR(15) REFERENCES credit_card(id),
175     business_id VARCHAR(20) REFERENCES company(company_id),
176     timestamp TIMESTAMP,
177     amount DECIMAL(10, 2),
178     declined BOOLEAN,
179     product_ids VARCHAR(50),
180     user_id INT REFERENCES user(id),
181     lat FLOAT,
182     longitude FLOAT,
183     discount_amount DECIMAL(10, 2),
184     tax_amount DECIMAL(10, 2),
185     shipping_amount DECIMAL(10, 2),
186     channel VARCHAR(50),
187     campaign_id VARCHAR(50),
188 );
```

The bottom output pane shows the execution results:

#	Time	Action	Message	Duration / Fetch
4	10:27:51	CREATE TABLE IF NOT EXISTS company (company_id VARCHAR(15) PRIMARY KEY, ...	0 row(s) affected	0.609 sec
5	10:28:40	CREATE TABLE IF NOT EXISTS user (id INT PRIMARY KEY, name VARCHAR(50), ...	0 row(s) affected	0.031 sec
6	10:29:45	CREATE TABLE IF NOT EXISTS credit_card (id VARCHAR(15) PRIMARY KEY, user_id IN...	0 row(s) affected	0.031 sec
7	10:30:54	CREATE TABLE IF NOT EXISTS transaction (id VARCHAR(255) PRIMARY KEY, card_id...	0 row(s) affected	0.031 sec

Ahora que tenemos las tablas creadas, empezamos el proceso de añadir los datos con USE transactions_star.

The screenshot shows the same SQL IDE window. The main editor displays the following SQL code:

```
-- Primero, creamos la base de datos
121 CREATE DATABASE IF NOT EXISTS transactions_star;
122
123 USE transactions_star;
```

The bottom output pane shows the execution results:

#	Time	Action	Message	Duration / Fetch
120	10:25:57	USE transactions_star	0 row(s) affected	0.000 sec
121	10:27:51	CREATE DATABASE IF NOT EXISTS transactions_star;	0 row(s) affected	0.000 sec
122	10:28:40	USE transactions_star;	0 row(s) affected	0.000 sec

Inserción de los datos en la tabla 'company':

The screenshot shows a SQL script being executed in SQL Server Enterprise Manager. The script defines columns for 'company' and then loads data from a CSV file. The output pane shows the execution results.

```
189     is_international BOOLEAN,  
190     decline_reason VARCHAR(100),  
191     distance_km DECIMAL(10, 2)  
192 );  
193  
194 -- Insertamos datos de companies  
195 • LOAD DATA  
196 INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__companies.csv'  
197 INTO TABLE company  
198 FIELDS TERMINATED BY ','  
199 IGNORE 1 ROWS;
```

Output

#	Time	Action	Message	Duration / Fetch
5	10:28:40	CREATE TABLE IF NOT EXISTS user (id INT PRIMARY KEY, name VARCHAR(50), ...	0 row(s) affected	0.031 sec
6	10:29:45	CREATE TABLE IF NOT EXISTS credit_card (id VARCHAR(15) PRIMARY KEY, user_id IN...	0 row(s) affected	0.031 sec
7	10:30:54	CREATE TABLE IF NOT EXISTS transaction (id VARCHAR(255) PRIMARY KEY, card_j...	0 row(s) affected	0.031 sec
8	10:34:43	LOAD DATA INFILE C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__com...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0	0.078 sec

Inserción de los datos en la tabla 'user':

The screenshot shows a SQL script being executed in SQL Server Enterprise Manager. The script defines columns for 'user' and then loads data from two CSV files. The output pane shows the execution results.

```
200  
201 -- Insertamos datos de users  
202 • LOAD DATA  
203 INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__american_users.csv'  
204 INTO TABLE user  
205 FIELDS TERMINATED BY ','  
206 ENCLOSED BY ''''  
207 IGNORE 1 ROWS;  
208  
209 • LOAD DATA  
210 INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__european_users.csv'  
211 INTO TABLE user  
212 FIELDS TERMINATED BY ','  
213 ENCLOSED BY ''''  
214 IGNORE 1 ROWS;  
215  
216
```

Output

#	Time	Action	Message	Duration / Fetch
7	10:30:54	CREATE TABLE IF NOT EXISTS transaction (id VARCHAR(255) PRIMARY KEY, card_j...	0 row(s) affected	0.031 sec
8	10:34:43	LOAD DATA INFILE C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__com...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0	0.078 sec
9	10:36:54	LOAD DATA INFILE C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__ame...	1010 row(s) affected Records: 1010 Deleted: 0 Skipped: 0 Warnings: 0	0.046 sec
10	10:36:57	LOAD DATA INFILE C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__euro...	3990 row(s) affected Records: 3990 Deleted: 0 Skipped: 0 Warnings: 0	0.141 sec

Inserción de los datos en la tabla 'credit_card':

The screenshot shows the SQL Server Enterprise Manager interface. The top pane displays a SQL script with the following commands:

```
-- Insertamos datos de credit cards
LOAD DATA
INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__credit_cards.csv'
INTO TABLE credit_card
FIELDS TERMINATED BY ','
IGNORE 1 ROWS;
```

The bottom pane shows the output of the execution, with a table listing the actions and their results:

#	Time	Action	Message	Duration / Fetch
8	10:34:43	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__com...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0	0.078 sec
9	10:36:54	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__ame...	1010 row(s) affected Records: 1010 Deleted: 0 Skipped: 0 Warnings: 0	0.046 sec
10	10:36:57	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__euro...	3990 row(s) affected Records: 3990 Deleted: 0 Skipped: 0 Warnings: 0	0.141 sec
11	10:37:27	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__cred...	5000 row(s) affected Records: 5000 Deleted: 0 Skipped: 0 Warnings: 0	0.172 sec

Inserción de los datos en la tabla 'transaction':

The screenshot shows the SQL Server Enterprise Manager interface. The top pane displays a SQL script with the following commands:

```
-- Insertamos datos de transactions
LOAD DATA
INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__transactions.csv'
INTO TABLE transaction
FIELDS TERMINATED BY ';'
IGNORE 1 ROWS;
```

The bottom pane shows the output of the execution, with a table listing the actions and their results:

#	Time	Action	Message	Duration / Fetch
9	10:36:54	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__ame...	1010 row(s) affected Records: 1010 Deleted: 0 Skipped: 0 Warnings: 0	0.046 sec
10	10:36:57	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__euro...	3990 row(s) affected Records: 3990 Deleted: 0 Skipped: 0 Warnings: 0	0.141 sec
11	10:37:27	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__cred...	5000 row(s) affected Records: 5000 Deleted: 0 Skipped: 0 Warnings: 0	0.172 sec
12	10:37:56	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__tran...	100000 row(s) affected Records: 100000 Deleted: 0 Skipped: 0 Warnings: 0	3.406 sec

Ahora con todos los datos inseridos, nos resta apenas crear la nueva columna 'region' en la tabla de usuarios:

```
-- Creamos la columna región en la tabla de usuarios
230
231 ALTER TABLE user
232 ADD region VARCHAR(100);
233
234 UPDATE user
235 SET region = CASE WHEN country = 'United States' THEN 'America'
236                 WHEN country = 'Canada' THEN 'America'
237                 ELSE 'Europa'
238             END
239 WHERE id >= 0;
240
```

id	country	region
1	United States	America
2	United States	America
3	United States	America
4	United States	America
5	United States	America

#	Time	Action	Message	Duration / Fetch
1	20:24:45	USE transactions_star	0 row(s) affected	0.000 sec
2	20:25:39	SELECT id, country, region FROM user	5000 row(s) returned	0.016 sec / 0.015 sec

Al comprobar el diagrama, las relaciones no aparecían como se esperaba, por lo que se añadieron explícitamente las claves foráneas mediante ALTER TABLE.

FK de transacciones:

```
-- Por fin, establecer las relaciones entre las tablas
238
239 ALTER TABLE transaction
240 ADD CONSTRAINT fk_card_id
241 FOREIGN KEY (card_id)
242 REFERENCES credit_card(id);
243
244 ALTER TABLE transaction
245 ADD CONSTRAINT fk_business_id
246 FOREIGN KEY (business_id)
247 REFERENCES company(company_id);
248
249 ALTER TABLE transaction
250 ADD CONSTRAINT fk_user_id
251 FOREIGN KEY (user_id)
252 REFERENCES user(id);
253
```

#	Time	Action	Message	Duration / Fetch
14	10:42:38	SELECT *, CASE WHEN country = 'United States' THEN 'America' WHEN country = 'Can...	5000 row(s) returned	0.000 sec / 0.016 sec
15	10:47:20	ALTER TABLE transaction ADD CONSTRAINT fk_card_id FOREIGN KEY (card_id) REFE...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0	2.797 sec
16	10:47:57	ALTER TABLE transaction ADD CONSTRAINT fk_business_id FOREIGN KEY (business_id)...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0	7.235 sec
17	10:48:40	ALTER TABLE transaction ADD CONSTRAINT fk_user_id FOREIGN KEY (user_id) REFE...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0	3.672 sec

FK de las tarjetas:

Tasca S2.01. Nocions bàsiques.

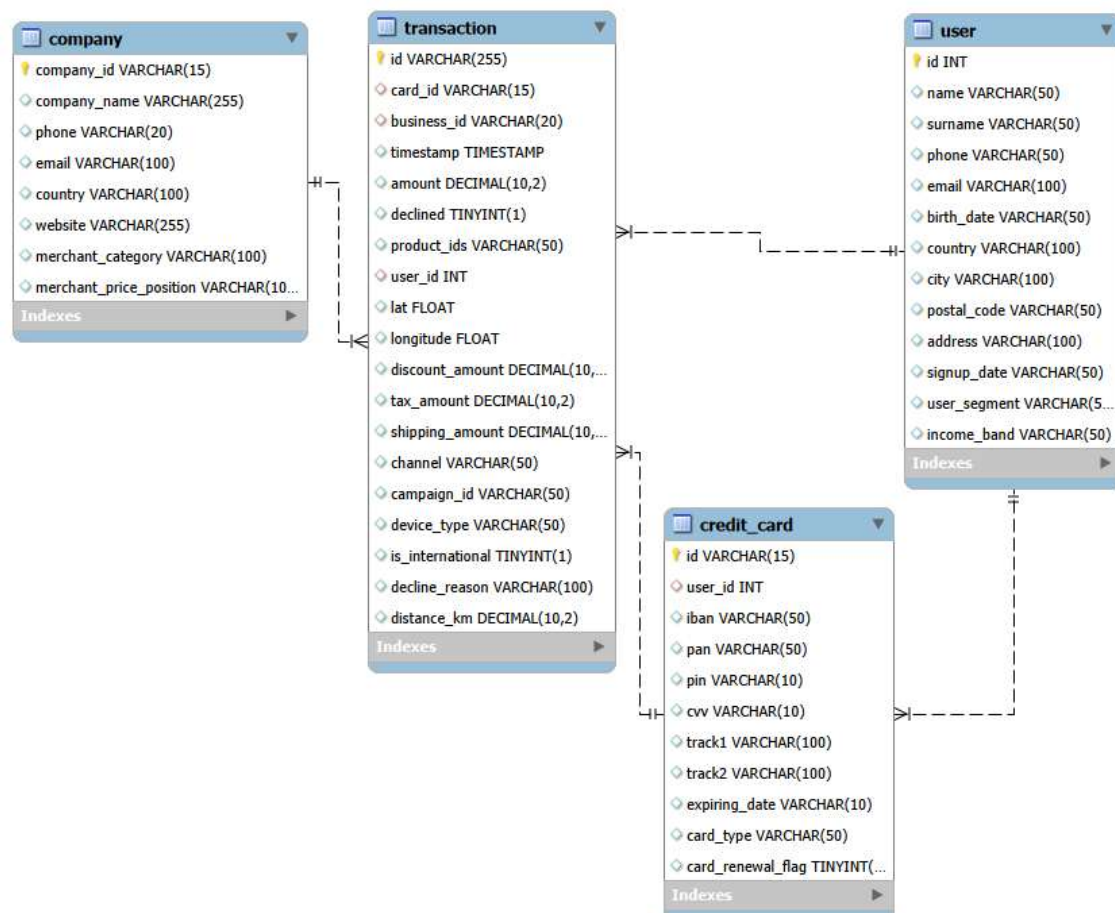
```

254 • ALTER TABLE credit_card
255 ADD CONSTRAINT fk_card_user_id
256 FOREIGN KEY (user_id)
257 REFERENCES user(id);
258
259
260
261
262
263
264
265
266
267
268
269

```

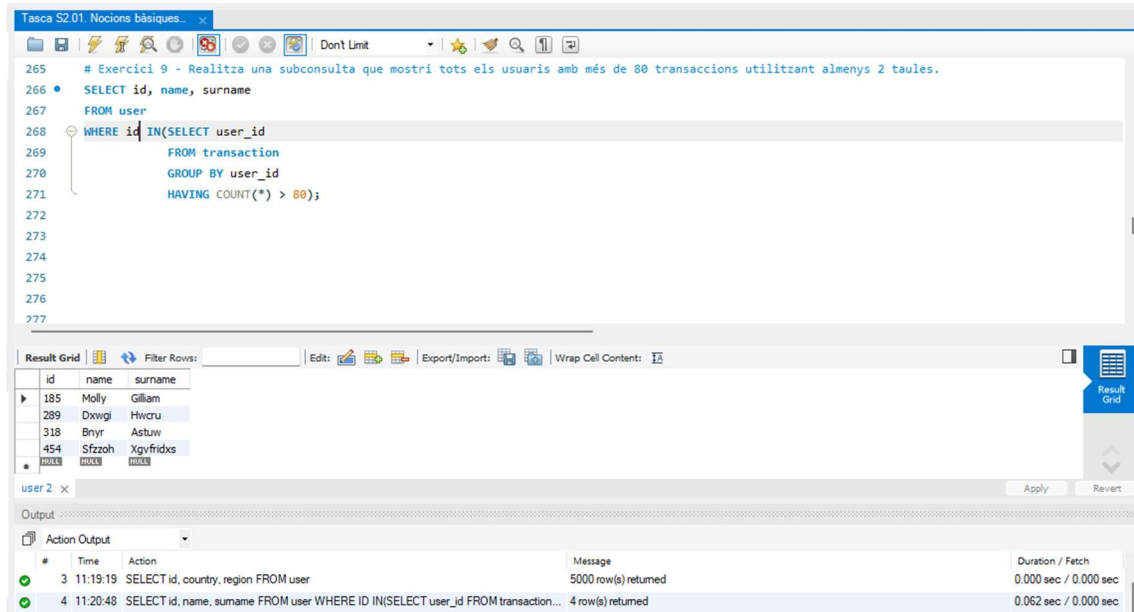
Output

#	Time	Action	Message	Duration / Fetch
16	10:47:57	ALTER TABLE transaction ADD CONSTRAINT fk_business_id FOREIGN KEY (business_id) REFERENCES business(id);	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0	7.235 sec
17	10:48:40	ALTER TABLE transaction ADD CONSTRAINT fk_user_id FOREIGN KEY (user_id) REFERENCES user(id);	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0	3.672 sec
18	10:50:56	ALTER TABLE credit_card ADD CONSTRAINT fk_user_id FOREIGN KEY (user_id) REFERENCES user(id);	Error Code: 1826. Duplicate foreign key constraint name 'fk_user_id'	0.000 sec
19	10:51:23	ALTER TABLE credit_card ADD CONSTRAINT fk_card_user_id FOREIGN KEY (user_id) REFERENCES user(id);	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0	0.313 sec



Exercici 9 - Realitza una subconsulta que mostri tots els usuaris amb més de 80 transaccions utilitzant almenys 2 taules.

Para llegar al resultado, la subconsulta devuelve los 'ids' de los usuarios que tienen más de 80 transacciones en la tabla. Cabe destacar que el filtro se aplica después de la agrupación por 'user_id', por eso se utiliza HAVING.



```
265 # Exercici 9 - Realitza una subconsulta que mostri tots els usuaris amb més de 80 transaccions utilitzant almenys 2 taules.
266 • SELECT id, name, surname
267 FROM user
268 WHERE id IN (SELECT user_id
269 FROM transaction
270 GROUP BY user_id
271 HAVING COUNT(*) > 80);
272
273
274
275
276
277
```

id	name	surname
185	Molly	Gillam
289	Dxwgi	Hwctu
318	Bnyr	Astuw
454	Sfzoh	Xgvfrids

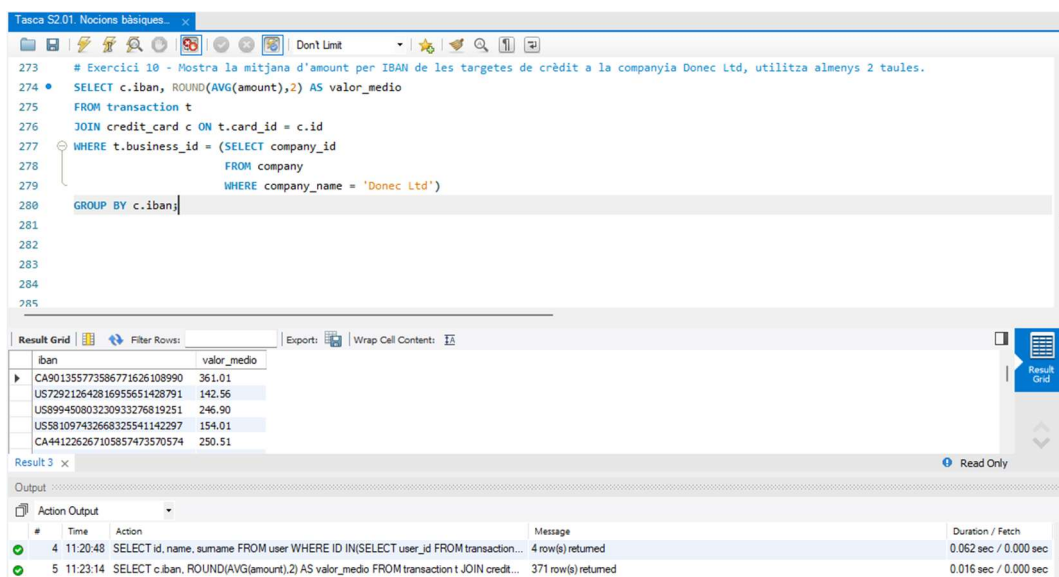
user 2 x

Output

#	Time	Action	Message	Duration / Fetch
3	11:19:19	SELECT id, country, region FROM user	5000 row(s) returned	0.000 sec / 0.000 sec
4	11:20:48	SELECT id, name, surname FROM user WHERE id IN (SELECT user_id FROM transaction...	4 row(s) returned	0.062 sec / 0.000 sec

Exercici 10 - Mostra la mitjana d'amount per IBAN de les targetes de crèdit a la companyia Donec Ltd, utilitza almenys 2 taules.

En este caso, opté por utilizar la subconsulta para obtener el código de la empresa. Una vez obtenido el id de la empresa Donec Ltd. se realizó un JOIN para relacionar las transacciones de la tabla 'transaction' con las tarjetas de crédito de la tabla 'credit_card' y obtener sus respectivos IBAN.



```
273 # Exercici 10 - Mostra la mitjana d'amount per IBAN de les targetes de crèdit a la companyia Donec Ltd, utilitza almenys 2 taules.
274 • SELECT c.iban, ROUND(AVG(amount),2) AS valor_medio
275 FROM transaction t
276 JOIN credit_card c ON t.card_id = c.id
277 WHERE t.business_id = (SELECT company_id
278 FROM company
279 WHERE company_name = 'Donec Ltd')
280 GROUP BY c.iban;
281
282
283
284
285
```

iban	valor_medio
CA901355773586771626108990	361.01
US729212642816955651428791	142.56
US899450803220933276819251	246.90
US581097432668325541142297	154.01
CA441226267105857473570574	250.51

Result 3 x

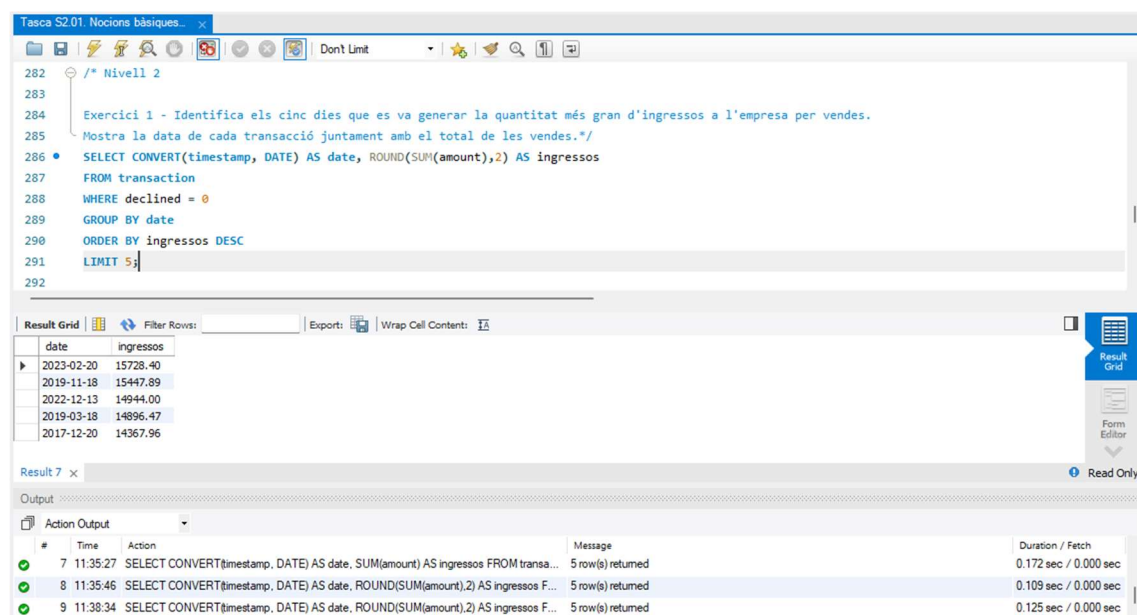
Output

#	Time	Action	Message	Duration / Fetch
4	11:20:48	SELECT id, name, surname FROM user WHERE id IN (SELECT user_id FROM transaction...	4 row(s) returned	0.062 sec / 0.000 sec
5	11:23:14	SELECT c.iban, ROUND(AVG(amount),2) AS valor_medio FROM transaction t JOIN credit...	371 row(s) returned	0.016 sec / 0.000 sec

Nivell 2

Exercici 1 - Identifica els cinc dies que es va generar la quantitat més gran d'ingressos a l'empresa per vendes. Mostra la data de cada transacció juntament amb el total de les vendes.

‘timestamp’ nos muestra la fecha y hora de cada transacción, pero necesitamos agrupar las transacciones por día. Para obtener únicamente la fecha, utilicé CONVERT y posteriormente agrupé los datos por fecha para obtener los cinco días con mayores ingresos. Cabe destacar que para este cálculo solo se consideran las transacciones no declinadas.



The screenshot shows a SQL IDE window titled 'Tasca S2.01. Nocions bàsiques...'. The query editor contains the following SQL code:

```
/* Nivell 2
Exercici 1 - Identifica els cinc dies que es va generar la quantitat més gran d'ingressos a l'empresa per vendes.
Mostra la data de cada transacció juntament amb el total de les vendes.*/
SELECT CONVERT(timestamp, DATE) AS date, ROUND(SUM(amount),2) AS ingressos
FROM transaction
WHERE declined = 0
GROUP BY date
ORDER BY ingressos DESC
LIMIT 5;
```

The results are displayed in a table with two columns: 'date' and 'ingressos'.

date	ingressos
2023-02-20	15728.40
2019-11-18	15447.89
2022-12-13	14944.00
2019-03-18	14896.47
2017-12-20	14367.96

The bottom panel shows the 'Action Output' with the following details:

#	Time	Action	Message	Duration / Fetch
7	11:35:27	SELECT CONVERT(timestamp, DATE) AS date, SUM(amount) AS ingressos FROM transa...	5 row(s) returned	0.172 sec / 0.000 sec
8	11:35:46	SELECT CONVERT(timestamp, DATE) AS date, ROUND(SUM(amount),2) AS ingressos F...	5 row(s) returned	0.109 sec / 0.000 sec
9	11:38:34	SELECT CONVERT(timestamp, DATE) AS date, ROUND(SUM(amount),2) AS ingressos F...	5 row(s) returned	0.125 sec / 0.000 sec

Exercici 2 - Presenta el nom, telèfon, país, data i amount, d'aquelles empreses que van realitzar transaccions amb un valor comprès entre 350 i 400 euros i en alguna d'aquestes dates: 29 d'abril del 2015, 20 de juliol del 2018 i 13 de març del 2024. Ordena els resultats de major a menor quantitat.

Así como en el ejercicio anterior, nuevamente tuve que utilizar la función CONVERT para obtener las fechas solicitadas en el enunciado. Establecí la relación entre las tablas transaction y company y, posteriormente, apliqué filtros para obtener únicamente las tres fechas solicitadas y las transacciones con un amount comprendido entre 350 y 400 euros.

Tasca S2.01. Nocions bàsiques...

```

293 /*Exercici 2 - Presenta el nom, telèfon, país, data i amount, d'aquelles empreses que van realitzar transaccions amb un valor comprès
294 entre 350 i 400 euros i en alguna d'aquestes dates: 29 d'abril del 2015, 20 de juliol del 2018 i 13 de març del 2024.
295 Ordena els resultats de major a menor quantitat.*/
296 SELECT c.company_name, c.phone, c.country, CONVERT(t.timestamp, DATE) AS date, t.amount
297 FROM transaction t
298 JOIN company c ON t.business_id = c.company_id
299 WHERE CONVERT(t.timestamp, DATE) IN ('2015-04-29', '2018-07-20', '2024-03-13')
300 AND t.amount BETWEEN 350 AND 400
301 ORDER BY t.amount DESC;
302
303

```

Result Grid

company_name	phone	country	date	amount
Fringilla LLC	+1 150 784 1716	New Zealand	2015-04-29	390.33
Auctor Mauris Vel LLP	+32 133 447 8239	United States	2024-03-13	383.58
Mauris Institute	+75 473 619 9472	Sweden	2015-04-29	369.20
Pede Cum Ltd	+26 625 798 6126	Norway	2018-07-20	358.64
Netus Et Malesuada Ltd	+52 818 402 1638	Netherlands	2024-03-13	352.21
Tempor Diam Institute	+89 848 141 3136	Netherlands	2024-03-13	350.71

Result 8

Output

#	Time	Action	Message	Duration / Fetch
8	11:35:46	SELECT CONVERT(timestamp, DATE) AS date, ROUND(SUM(amount),2) AS ingresos F...	5 row(s) returned	0.109 sec / 0.000 sec
9	11:38:34	SELECT CONVERT(timestamp, DATE) AS date, ROUND(SUM(amount),2) AS ingresos F...	5 row(s) returned	0.125 sec / 0.000 sec
10	11:41:35	SELECT c.company_name, c.phone, c.country, CONVERT(t.timestamp, DATE) AS date, t...	6 row(s) returned	0.078 sec / 0.000 sec

Exercici 3 - Necessitem optimitzar l'assignació dels recursos i dependrà de la capacitat operativa que es requereixi, per la qual cosa et demanen la informació sobre la quantitat de transaccions que realitzen les empreses, però el departament de recursos humans és exigent i vol un llistat de les empreses on especifiqueu si tenen igual o més de 400 transaccions o menys.

Para clasificar las empresas según su capacidad operativa, utilicé CASE. De esta manera, podemos visualizar de forma sencilla la cantidad de transacciones y su correspondiente clasificación en una única tabla.

Tasca S2.01. Nocions bàsiques...

```

303 /*Exercici 3 - Necessitem optimitzar l'assignació dels recursos i dependrà de la capacitat operativa que es requereixi, per la qual cosa
304 et demanen la informació sobre la quantitat de transaccions que realitzen les empreses, però el departament de recursos humans és exigent
305 i vol un llistat de les empreses on especifiqueu si tenen igual o més de 400 transaccions o menys.*/
306 SELECT t.business_id, c.company_name, COUNT(*) as num_transacciones,
307 CASE WHEN COUNT(*) >= 400 THEN '400 o más'
308 ELSE 'menos de 400'
309 END AS capacidad_operativa
310 FROM transaction t
311 JOIN company c ON t.business_id = c.company_id
312 GROUP BY t.business_id, c.company_name;
313

```

Result Grid

business_id	company_name	num_transacciones	capacidad_operativa
b-2222	Ac Fermentum Incorporated	2401	400 o más
b-2226	Magna A Neque Industries	410	400 o más
b-2230	Fusce Corp.	447	400 o más
b-2234	Convallis In Incorporated	1514	400 o más
b-2238	Ante Jaculis Nec Foundation	472	400 o más
b-2242	Donec Ltd	449	400 o más

Result 10

Output

#	Time	Action	Message	Duration / Fetch
10	11:41:35	SELECT c.company_name, c.phone, c.country, CONVERT(t.timestamp, DATE) AS date, t...	6 row(s) returned	0.078 sec / 0.000 sec
11	11:46:00	SELECT t.business_id, c.company_name, COUNT(*) as num_transacciones, CASE WHEN...	100 row(s) returned	0.218 sec / 0.000 sec
12	11:48:43	SELECT t.business_id, c.company_name, COUNT(*) as num_transacciones, CASE WHEN...	100 row(s) returned	0.156 sec / 0.000 sec

Exercici 4 - Elimina de la taula transaction el registre amb ID 000447FE-B650-4DCF-85DE-C7ED0EE1CAAD de la base de dades.

Antes:

The screenshot shows a SQL IDE window titled 'Tasca S2.01. Nocions bàsiques...'. The SQL editor contains the following code:

```
314 # Exercici 4 - Elimina de la taula transaction el registre amb ID 000447FE-B650-4DCF-85DE-C7ED0EE1CAAD de la base de dades.
315 SELECT *
316 FROM transaction
317 WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
318
319
320
321
322
323
324
```

The 'Result Grid' at the bottom displays a single row of data for the transaction with ID '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'. The columns include id, card_id, business_id, timestamp, amount, declined, product_ids, user_id, lat, longitude, discount_amount, tax_amount, and shipping_amount.

The 'Output' pane shows the execution log for 'transaction 11':

#	Time	Action	Message	Duration / Fetch
11	11:46:00	SELECT t.business_id, c.company_name, COUNT(*) as num_transacciones, CASE WHEN...	100 row(s) returned	0.218 sec / 0.000 sec
12	11:48:43	SELECT t.business_id, c.company_name, COUNT(*) as num_transacciones, CASE WHEN...	100 row(s) returned	0.156 sec / 0.000 sec
13	11:50:49	SELECT * FROM transaction WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'	1 row(s) returned	0.016 sec / 0.000 sec

Utilizamos el comando DELETE para eliminar la transacción asociada al id indicado. Finalmente, realizamos una consulta para comprobar que el registro ha sido eliminado correctamente.

The screenshot shows the same SQL IDE window after executing the DELETE statement. The SQL editor now contains:

```
316 FROM transaction
317 WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
318
319 DELETE
320 FROM transaction
321 WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
322
323 SELECT *
324 FROM transaction
325 WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
326
```

The 'Result Grid' now displays a table with 14 columns: id, card_id, business_id, timestamp, amount, declined, product_ids, user_id, lat, longitude, discount_amount, tax_amount, shipping_amount, channel, campaign_id, and device_type. The row for the specified ID is no longer present.

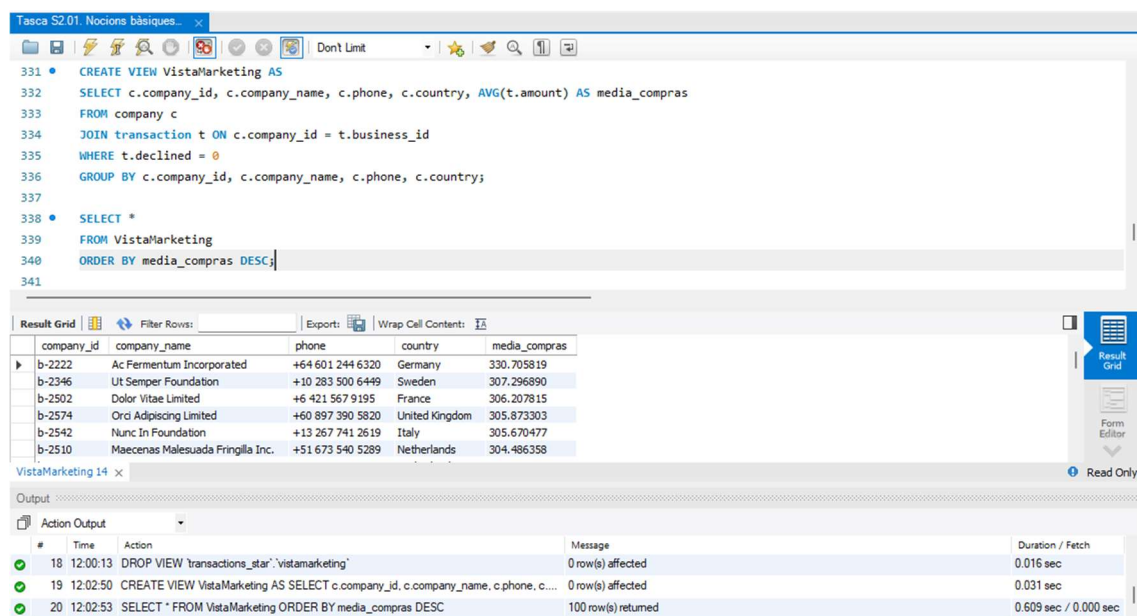
The 'Output' pane shows the execution log for 'transaction 12':

#	Time	Action	Message	Duration / Fetch
13	11:50:49	SELECT * FROM transaction WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'	1 row(s) returned	0.016 sec / 0.000 sec
14	11:51:17	DELETE FROM transaction WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'	1 row(s) affected	0.015 sec
15	11:51:20	SELECT * FROM transaction WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'	0 row(s) returned	0.000 sec / 0.000 sec

Exercici 5 - La secció de màrqueting desitja tenir accés a informació específica per a realitzar anàlisi i estratègies efectives. S'ha sol·licitat crear una vista que proporcioni detalls clau sobre les companyies i les seves transaccions. Serà necessària que creïs una vista anomenada VistaMarketing que contingui la

següent informació: Nom de la companyia. Telèfon de contacte. País de residència. Mitjana de compra realitzat per cada companyia. Presenta la vista creada, ordenant les dades de major a menor mitjana de compra.

El primer pas es crear la vista, o VIEW. El procés és similar a una consulta tradicional con SELECT, pero permet guardar la consulta para poder accedir posteriormente a esta información de forma más sencilla. Fue necesario realizar una agrupación para obtener la media de las compras de cada compañía, excluyendo las transacciones declinadas. Finalmente, presentamos la vista ordenada de mayor a menor media de compras.



```
331 CREATE VIEW VistaMarketing AS
332 SELECT c.company_id, c.company_name, c.phone, c.country, AVG(t.amount) AS media_compras
333 FROM company c
334 JOIN transaction t ON c.company_id = t.business_id
335 WHERE t.declined = 0
336 GROUP BY c.company_id, c.company_name, c.phone, c.country;
337
338 SELECT *
339 FROM VistaMarketing
340 ORDER BY media_compras DESC;
```

company_id	company_name	phone	country	media_compras
b-2222	Ac Fermentum Incorporated	+64 601 244 6320	Germany	330.705819
b-2346	Ut Semper Foundation	+10 283 500 6449	Sweden	307.296890
b-2502	Dolor Vitae Limited	+6 421 567 9195	France	306.207815
b-2574	Orci Adipiscing Limited	+60 897 390 5820	United Kingdom	305.873303
b-2542	Nunc In Foundation	+13 267 741 2619	Italy	305.670477
b-2510	Maeenas Malesuada Fringilla Inc.	+51 673 540 5289	Netherlands	304.486358

#	Time	Action	Message	Duration / Fetch
18	12:00:13	DROP VIEW 'transactions_star'.vistamarketing	0 row(s) affected	0.016 sec
19	12:02:50	CREATE VIEW VistaMarketing AS SELECT c.company_id, c.company_name, c.phone, c...	0 row(s) affected	0.031 sec
20	12:02:53	SELECT * FROM VistaMarketing ORDER BY media_compras DESC	100 row(s) returned	0.609 sec / 0.000 sec

Nivell 3

Exercici 1 - Crea una nova taula que reflecteixi l'estat de les targetes de crèdit basat en si les tres últimes transaccions han estat declinades aleshores és inactiu, si almenys una no és rebutjada aleshores és actiu. Partint d'aquesta taula respon:

Para obtener las tres transacciones más recientes de cada tarjeta, fue necesario utilizar la función de ventana ROW_NUMBER() junto con PARTITION BY, una función que todavía no había utilizado. Esto permitió ordenar las transacciones de cada tarjeta según su fecha y seleccionar las tres más recientes. A partir de estas transacciones, se utilizó SUM(declined) para clasificar las tarjetas como activas o inactivas y, finalmente, crear la tabla solicitada.

Tasca S2.01. Nocions bàsiques...

```

342 /* Nivell 3
343
344 Exercici 1 - Crea una nova taula que reflecteixi l'estat de les targetes de crèdit basat en si les tres últimes transaccions han estat
345 declinades aleshores és inactiu, si almenys una no és rebutjada aleshores és actiu. Partint d'aquesta taula respon:*/
346 CREATE TABLE IF NOT EXISTS estado_tarjetas AS
347     SELECT id AS card_id,
348         CASE WHEN SUM(declined) = 3 THEN 'Inactivo'
349             ELSE 'Activo'
350         END AS estado_tarjeta
351     FROM (SELECT c.id, t.timestamp, t.declined, ROW_NUMBER() OVER (PARTITION BY c.id
352         ORDER BY t.timestamp DESC) AS position
353         FROM credit_card c
354         JOIN transaction t ON c.id = t.card_id) AS filtered_table
355     WHERE position IN (1, 2, 3)
356     GROUP BY id;
357
358
359
360

```

Output

#	Time	Action	Message	Duration / Fetch
19	12:02:50	CREATE VIEW VistaMarketing AS SELECT c.company_id, c.company_name, c.phone, c...	0 row(s) affected	0.031 sec
20	12:02:53	SELECT * FROM VistaMarketing ORDER BY media_compras DESC	100 row(s) returned	0.609 sec / 0.000 sec
21	12:04:36	CREATE TABLE IF NOT EXISTS estado_tarjetas AS SELECT id AS card_id, CASE WHE...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0	1.422 sec

👉 Quantes targetes estan actives?

Con la tabla 'estado_tarjetas' ya creada, utilizamos COUNT(*) para obtener el número de tarjetas cuyo estado es Activo.

Tasca S2.01. Nocions bàsiques...

```

358 # Quantes targetes estan actives?
359 SELECT COUNT(*) AS tarjetas_activas
360 FROM estado_tarjetas
361 WHERE estado_tarjeta = 'Activo';
362
363
364
365
366
367
368

```

Result Grid

tarjetas_activas
5000

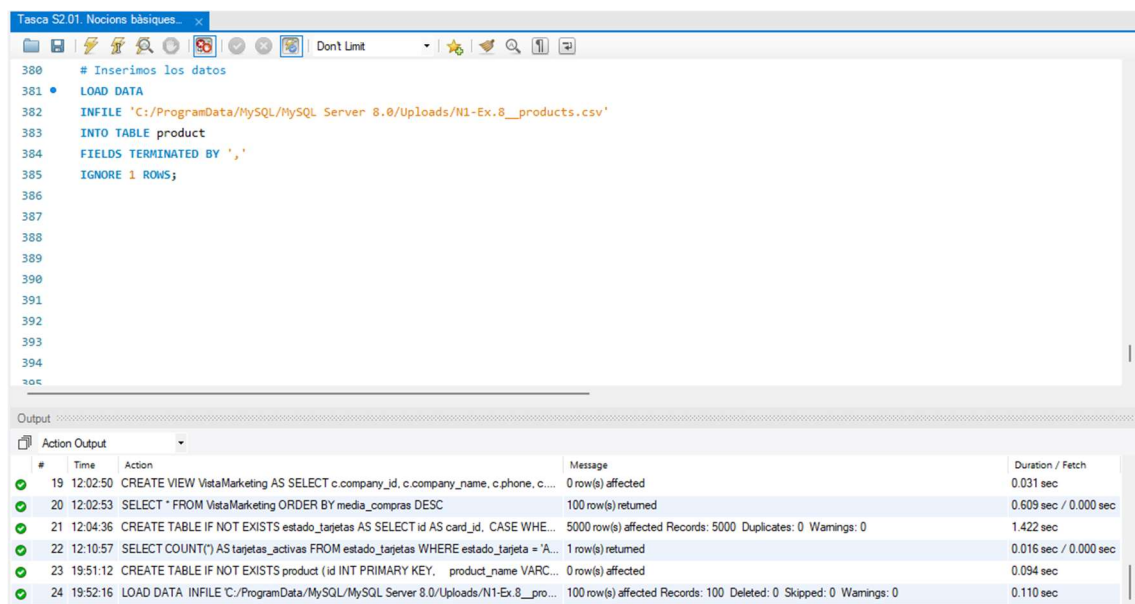
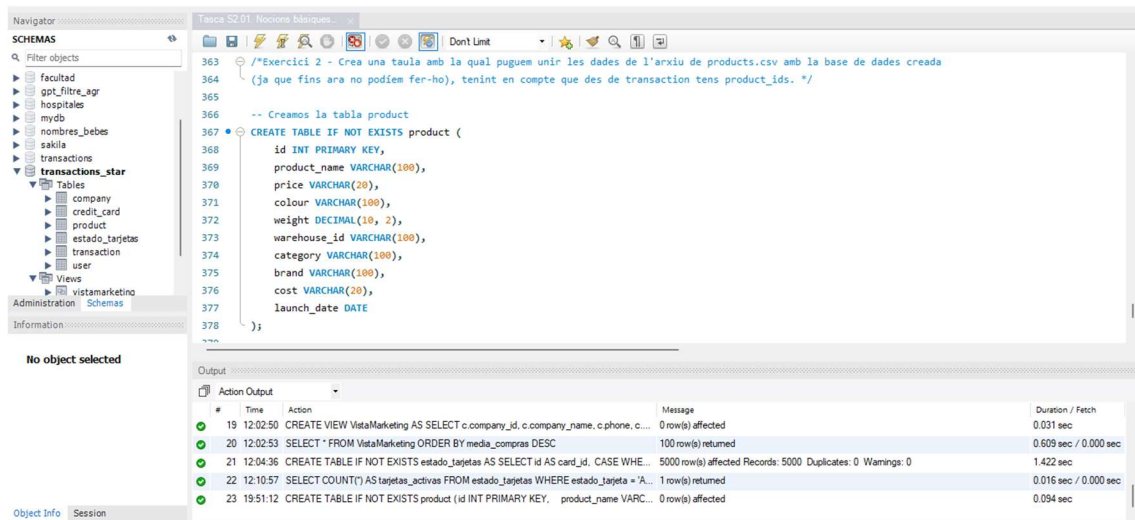
Result 15 x

Output

#	Time	Action	Message	Duration / Fetch
20	12:02:53	SELECT * FROM VistaMarketing ORDER BY media_compras DESC	100 row(s) returned	0.609 sec / 0.000 sec
21	12:04:36	CREATE TABLE IF NOT EXISTS estado_tarjetas AS SELECT id AS card_id, CASE WHE...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0	1.422 sec
22	12:10:57	SELECT COUNT(*) AS tarjetas_activas FROM estado_tarjetas WHERE estado_tarjeta = 'A...	1 row(s) returned	0.016 sec / 0.000 sec

Exercici 2 - Crea una taula amb la qual puguem unir les dades de l'arxiu de products.csv amb la base de dades creada (ja que fins ara no podíem fer-ho), tenint en compte que des de transaction tens product_ids.

Otra vez vamos a crear la tabla de productos con CREATE TABLE IF NOT EXISTS y luego cargar los datos con LOAD DATA INFILE.



Para que podamos hacer la conexión con la tabla de transacciones y mantener los ids de las transacciones además de cada id de producto, utilicé la función `FIND_IN_SET()` por ser la más sencilla. Como en Python, pasamos el valor de `p.id` y solicitamos que la función lo encuentre en una columna que se comporta como una lista. Además, fue necesario utilizar `REPLACE()` para eliminar los espacios y evitar problemas en la búsqueda.

Navigator

SCHMAS

Filter objects

- facultad
- got_filtre_agr
- hospitales
- mydb
- nombres_bebes
- sakila
- transactions
- transactions_star
 - Tables
 - company
 - credit_card
 - join_transaction_produ
 - product
 - estado_tarjetas
 - transaction
 - user
 - Views

Administration Schemas

Information

No object selected

Tasca S2.01. Nocions bàsiques...

Don't Limit

```

387 -- Creamos la tabla intermedia
388 CREATE TABLE join_transaction_product AS
389 SELECT t.id, t.card_id, t.business_id, t.timestamp, t.amount, t.declined, t.user_id, t.lat, t.longitude, t.discount_amount, t.tax_amount, t.shipping_
390 FROM transaction t
391 JOIN product p ON FIND_IN_SET(p.id, REPLACE(t.product_ids, ' ', '')) > 0;
392
393
394
395
396
397
398
399
400
401
402
403
404

```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
31	20:08:50	CREATE TABLE join_transaction_product AS SELECT t.id, t.card_id, t.business_id, t.timestamp, t.amount, t.declined, t.user_id, t.lat, t.longitude, t.discount_amount, t.tax_amount, t.shipping_	Error Code: 1050. Table 'join_transaction_product' already exists	0.000 sec
32	20:09:27	DROP TABLE transactions_star; join_transaction_product	0 row(s) affected	0.062 sec
33	20:09:30	CREATE TABLE join_transaction_product AS SELECT t.id, t.card_id, t.business_id, t.timestamp, t.amount, t.declined, t.user_id, t.lat, t.longitude, t.discount_amount, t.tax_amount, t.shipping_	253388 row(s) affected Records: 253388 Duplicates: 0 Warnings: 0	12.500 sec

Object Info Session

Genera la següent consulta:

👉 Necessitem conèixer el nombre de vegades que s'ha venut cada producte.

Con la nueva tabla, hacemos un conteo de ventas por producto, aplicando el filtro en la cláusula WHERE para considerar solamente las transacciones no declinadas.

Tasca S2.01. Nocions bàsiques...

Don't Limit

```

392
393 /* Genera la següent consulta:
394
395 Necessitem conèixer el nombre de vegades que s'ha venut cada producte.*/
396 SELECT product_id, COUNT(*) AS ventas_total
397 FROM join_transaction_product
398 WHERE declined = 0
399 GROUP BY product_id;

```

Result Grid

product_id	ventas_total
97	2514
87	2575
26	2516
16	2583
81	2451
30	2440
11	2556
72	2550

Result 21

Output

Action Output

#	Time	Action	Message	Duration / Fetch
34	20:10:11	SELECT p.id, COUNT(*) FROM transaction t JOIN product p ON FIND_IN_SET(p.id, REPL...	100 row(s) returned	3.484 sec / 0.000 sec
35	20:11:33	SELECT product_id, COUNT(*) FROM join_transaction_product WHERE declined = 0 GR...	100 row(s) returned	0.281 sec / 0.000 sec
36	20:13:19	SELECT product_id, COUNT(*) AS ventas_total FROM join_transaction_product WHERE ...	100 row(s) returned	0.234 sec / 0.000 sec